Name of the Student	**V Sunil Kumar**
Reg. No	**192425292**
GitHub Link	**https://github.com/192425292simats-byte/MLA0403-DEEP-LEARNING**

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 1

Design and Develop Model

COURSE NAME: Deep Learning **COURSE CODE: MLA0403**
SLOT : A

COURSE OUTCOME COVERED

CO 4: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. BL-6

Course Outcome Weightage: 10% **Duration: 90 minutes**
Assessment Tool Weightage: 30% **Mark : 25**

Code of Conduct:
I (V Sunil Kumar / 192425292) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Name of the student:V Sunil**

Criteria	Problem Understanding & Requirement Analysis **(2)**	Model Architecture & Design **(2.5)**	Application of Deep Learning Techniques **(2.5)**	Model Development & Implementation Strategy **(2)**	Performance Analysis & Justification ***(1)***	**Total (10)**
Weightage	***20 %***	***30%***	***25%***	***15 %***	***10 %***	**100 %**
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

QUESTIONS: 05 Total Marks:25

Q1. Transfer Learning using ResNet for X-ray Classification

1. Problem Understanding and Requirement Analysis

The hospital has only 2,000 labeled X-ray images for classifying images into Normal and Abnormal categories. This is a relatively small dataset for training a deep CNN from scratch. A pre-trained ResNet model can therefore be reused to obtain general visual features and reduce overfitting.

2. Model Architecture and Design

A suitable model is ResNet-50 pre-trained on ImageNet. The proposed architecture is: X-ray Image → ResNet-50 Feature Extractor → Global Average Pooling → Dense Layer → Dropout → Sigmoid Output. The input X-ray image is resized to 224 × 224 pixels. The original ResNet classification layer is removed and replaced with a new Dense layer containing one neuron with sigmoid activation for binary classification.

3. Frozen and Fine-Tuned Layers

Initially, the early convolutional layers and most middle layers of ResNet should be frozen because they contain general features such as edges, textures and shapes. The original final classification layer is replaced. The new classification layer is trained first. After initial convergence, the last residual block or a few deeper layers can be unfrozen and fine-tuned using a small learning rate. This allows the model to learn X-ray-specific features while reducing overfitting.

4. Model Development and Implementation Strategy

Preprocess the X-ray images by resizing and normalizing them. Use suitable data augmentation such as small rotations, zooming and shifting, while avoiding transformations that could change clinical meaning. Split the dataset into training, validation and test sets. Use binary cross-entropy loss and an optimizer such as Adam. Early stopping and learning-rate scheduling can further improve generalization.

5. Performance Analysis and Justification

Evaluate the model using accuracy, precision, recall or sensitivity, specificity, F1-score, confusion matrix and ROC-AUC. In medical diagnosis, sensitivity is especially important because failing to identify an abnormal X-ray can be serious. Compare the transfer-learning model with a model trained from scratch to validate the benefit of pre-training.

6. Conclusion

Transfer learning with ResNet is an effective solution for the hospital because it uses previously learned visual representations. Freezing most layers initially and fine-tuning only deeper layers

provides a practical balance between classification performance, training time and overfitting control.

Q2. DenseNet for Plant Disease Classification

1. Problem Understanding and Requirement Analysis

The agricultural organization wants to classify leaf images into five plant disease categories. Because the dataset contains limited training samples, training a deep CNN from scratch may overfit. DenseNet is suitable because its dense connections promote feature reuse and efficient information flow.

2. Model Architecture and Design

A suitable model is DenseNet-121. The architecture is: Leaf Image → DenseNet Feature Extractor → Global Average Pooling → Dropout → Dense Layer → Softmax Output. The final classification layer contains five neurons, representing the five disease classes.

3. Dense Connections and Feature Reuse

In a conventional CNN, a layer normally receives information mainly from the previous layer. In DenseNet, each layer within a dense block receives feature maps from all preceding layers. Thus, earlier features such as edges, textures and leaf patterns remain directly available to later layers. This reduces repeated feature learning and improves information and gradient flow.

4. Model Development and Implementation Strategy

Resize and normalize the leaf images and apply augmentation such as rotation, scaling and flipping. Load a pre-trained DenseNet-121 and initially freeze its feature-extraction layers. Remove the original classifier and add Global Average Pooling, Dropout and a five-neuron Softmax layer. Train the new classifier first and then fine-tune the final DenseNet block using a low learning rate.

5. Performance Analysis and Justification

Evaluate accuracy, precision, recall, F1-score, confusion matrix and per-class accuracy. The confusion matrix can identify which diseases are commonly confused. Validation loss and training-validation curves can also be monitored to detect overfitting.

6. Conclusion

DenseNet is highly suitable for limited plant-image datasets because dense connections allow effective feature reuse, improve gradient propagation and reduce unnecessary learning. Combined with transfer learning, it can provide accurate and efficient plant disease classification.

Q3. LSTM for Student Academic Performance Prediction

1. Problem Understanding and Requirement Analysis

The university wants to predict a student's future academic performance using weekly attendance, assignment marks, internal assessment marks and previous examination scores. Because these measurements are collected over several weeks, the data has a temporal or sequential nature. LSTM is appropriate because it can learn relationships across time.

2. Model Architecture and Design

The proposed architecture is: Weekly Student Data → LSTM Layer → Dropout → Dense Layer → Softmax Output. For each week, the input can be represented as [Attendance, Assignment Mark, Internal Assessment Mark, Previous Examination Score]. The sequence is supplied to the LSTM in chronological order.

3. Role of LSTM

LSTM uses a cell state and three major gates. The forget gate determines which previous information should be discarded, the input gate determines which new information should be stored, and the output gate determines which information is passed forward. This allows the model to remember important academic patterns from earlier weeks.

4. Model Development and Implementation Strategy

Clean the student records, handle missing values and normalize numerical features. Convert the weekly observations into fixed-length sequences and divide them into training, validation and test sets. A practical architecture can use an LSTM layer followed by Dropout, a ReLU Dense layer and a Softmax output layer. Categorical cross-entropy can be used as the loss function. Early stopping can reduce overfitting.

5. Performance Analysis and Justification

Evaluate the model using accuracy, precision, recall, F1-score and a confusion matrix. The predicted performance category should be compared with the actual final category. Performance can also be compared with a non-sequential baseline such as logistic regression or a basic feed-forward neural network.

6. Conclusion

LSTM is appropriate because academic performance data is time-dependent. Instead of treating each week's information independently, LSTM remembers useful information from previous weeks and uses it to predict the student's future performance category.

Q4. Bidirectional RNN for Customer Sentiment Classification

1. Problem Understanding and Requirement Analysis

The organization needs to classify customer reviews into Positive, Negative and Neutral categories. The meaning of a word may depend on both preceding and following words. A Bidirectional RNN is therefore suitable because it processes the sequence in both directions.

2. Major Processing Stages

The complete pipeline is: Review Text → Tokenization → Padding → Word Embedding → Bidirectional RNN → Dropout → Dense Layer → Softmax → Sentiment Prediction.

3. Working of the Model

First, the raw review is tokenized into words or subwords. The tokens are converted into numerical indices and padded to a common length. An embedding layer converts these indices into dense vectors. The Bidirectional RNN then processes the sequence from left to right and right to left. The two contextual representations are combined before classification.

4. Model Architecture and Development Strategy

A practical architecture is Embedding → Bidirectional LSTM or GRU → Dropout → Dense ReLU layer → three-neuron Softmax layer. Although a basic Bidirectional RNN can be used, Bidirectional LSTM or GRU is often preferred in practical applications because it handles longer dependencies more effectively. The text dataset should be divided into training, validation and test sets. Categorical cross-entropy can be used for training.

5. Performance Analysis and Justification

Evaluate the classifier using accuracy, precision, recall, F1-score and a confusion matrix. The confusion matrix is especially useful for identifying confusion between neutral and positive or negative reviews. Training and validation curves can be monitored to detect overfitting.

6. Conclusion

A Bidirectional RNN considers contextual information from both past and future words. This improves the understanding of word meaning and makes the architecture suitable for three-class customer sentiment classification.

Q5. Encoder–Decoder Model for English-to-Hindi Translation

1. Problem Understanding and Requirement Analysis

The software company needs to translate English sentences into Hindi. The input and output sentences can have different lengths and may have different word orders. An Encoder–Decoder sequence-to-sequence model is suitable because it can process one sequence and generate another sequence of different length.

2. Model Architecture

The architecture is: English Sentence → Encoder → Hidden/Context Representation → Decoder → Hindi Sentence. LSTM or GRU networks can be used in both the encoder and decoder.

3. Role of the Encoder

The encoder receives the English sentence one word at a time. Each word is converted into an embedding vector and processed sequentially. The encoder captures important information about the complete input sentence in its hidden state or context representation.

4. Role of Hidden Representation

The hidden or context representation acts as a bridge between the encoder and decoder. It contains the learned information needed by the decoder to generate the Hindi translation. In more advanced systems, attention can be used so that the decoder can access different encoder states instead of relying on one fixed representation.

5. Role of the Decoder and Output Layer

The decoder generates the Hindi sentence one word at a time. It uses its previous hidden state, the encoded information and the previously generated word to predict the next word. The final output layer is a Dense layer with Softmax activation. If the Hindi vocabulary contains 10,000 words, the output layer produces probabilities for approximately 10,000 possible words at each time step.

6. Model Development and Training Strategy

Use a parallel English-Hindi dataset and divide it into training, validation and test sets. Tokenize both languages, create vocabulary mappings and pad sequences where required. During training, the English sentence is passed to the encoder and the target Hindi sentence is supplied to the decoder. Categorical cross-entropy is used to calculate prediction error. Teacher forcing can be used by providing the correct previous Hindi word to the decoder during training.

7. Performance Analysis and Justification

Evaluate the model using validation loss, token-level accuracy and BLEU score. Human evaluation can also assess fluency, adequacy and correctness. Attention mechanisms and beam search can be added to improve translation quality.

8. Conclusion

The Encoder–Decoder model is suitable because the encoder understands the English input sequence while the decoder generates the Hindi output sequence step by step. It can handle different input and output lengths and is therefore well suited for machine translation.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Requirement Analysis	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Model Architecture & Design	30%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Application of Deep Learning Techniques	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Model Development & Implementation Strategy	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Performance Analysis & Justification	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis